

Capstone Project:  
Design, Build, and Defend a Secure Mobile App  
Jaxon Doolittle & Keenan Johnson  
GSW University

7/28//2026

## **App Overview**

**App Name:** Project Aegis (from Greek mythology)

**Core Concept:** A highly secure, offline-first mobile password manager and digital vault for Android.

**Target Users:** Security-conscious individuals who want to store credentials locally without relying on vulnerable third-party cloud servers.

### **General Functionality & Features:**

- **Biometric Gatekeeping:** The app is completely locked down and requires a master PIN or native Android fingerprint authentication to even open the dashboard.
- **Encrypted Storage:** Stores standard credential data (Account Name, URL, Username, Password) in a heavily encrypted local SQLite database.
- **Masked UI:** The main dashboard hides all passwords by default until the user explicitly toggles a visibility icon or copies them.
- **Anti-Scraping Clipboard:** When a user copies a password to paste it into another app, Project Aegis triggers a background timer that wipes the Android clipboard cache after 30 seconds to prevent malicious apps from scraping it.
- **Zero-Knowledge Sync (Optional/Stretch Goal):** If we use Firebase, we only push the fully encrypted ciphertext to the cloud, meaning the server never actually sees the plaintext passwords.

### **Team Roles:**

Jaxon Doolittle: Architecture & implementation

Keenan Johnson: Security analysis & threat modeling

## Architecture & Component List

### Core Components:

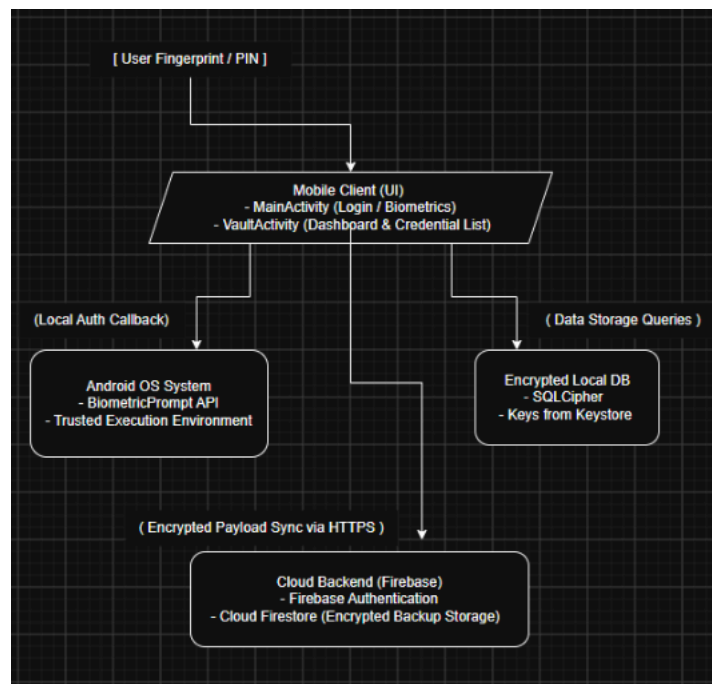
Mobile Client/Frontend UI: A native Android application written in Java

Local Storage Mechanism: Encrypted SQLCipher database system + Android Keystore system

External Android System Services: BiometricPrompt via Android's Trusted Execution Environment + Android's ClipBoardManager.

Cloud Backend/Database: FireBase, specifically using Cloud Firestore + Firebase Authentication

### Architecture Diagram



**Threat Modeling Table (Initial)**

| <b><u>Component/<br/>Boundary</u></b> | <b><u>STRIDE<br/>Category</u></b>        | <b><u>Threat</u></b>                                                                                       | <b><u>Potential Impact</u></b>                                                             | <b><u>Proposed Mitigation or<br/>Defense</u></b>                                                                                                                         |
|---------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cloud Sync                            | Tampering /<br>Information<br>Disclosure | MitM interceptor attempts to capture or alter sync traffic passing to Firebase Cloud Firestore             | Eavesdropping on communications or interception of system payloads                         | Enforce TLS 1.3 for all web calls; apply strict certificate pinning within the network security configuration file to block custom root certificates.                    |
| Android Clipboard                     | Information<br>Disclosure                | A background application monitors the Android clipboard system, scraping the raw password.                 | Exposure of highly sensitive plaintext credentials to unauthorized third-party apps.       | Implement a custom background handler to wipe clipboard cache after a short amount of time or after pasting.                                                             |
| Firebase Backup Storage               | Information<br>Disclosure                | Compromise of the Firebase console or cloud server infrastructure leaks the backed-up database records.    | Mass exposure of stored data payloads to external actors on the public internet.           | Maintain strict client-side zero-knowledge architecture. Database records are fully encrypted using client-managed AES-256 keys before transport.                        |
| Android OS Interface                  | Elevation of<br>Privilege                | Exploits inside older OS kernels allow processes to break isolation boundaries and read memory footprints. | Attacker extracts master keys or plaintext vault variables straight out of runtime memory. | Implement FLAG_SECURE across all UI Activities to prevent background screen scraping and OS snapshots. Clear password variables from RAM arrays immediately after usage. |
| Local Device Cache                    | Tampering                                | A malicious app with elevated privileges modifies the local SQLCipher file structure or configuration.     | Data corruption, loss of vault accessibility, or localized denial of service.              | Enforce rigid file permissions using internal application storage. Verify structural integrity parameters using SQLCipher internal health checks                         |

### Threat Modeling Table (After)

| <u>Component/<br/>Boundary</u>    | <u>STRIDE<br/>Category</u> | <u>Threat</u>                                                                                                                                                                                     | <u>Potential Impact</u>                                                                    | <u>Proposed Mitigation or<br/>Defense</u>                                                                                                                                                                                                         |
|-----------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local Device<br>Cache             | Tampering                  | A malicious app with elevated OS privileges bypasses Android sandbox controls and modifies the local SQLCipher file structure or configuration.                                                   | Data corruption, loss of vault accessibility, or localized denial of service.              | Enforce rigid file permissions using internal application storage. Verify structural integrity parameters using SQLCipher internal health checks, and incorporate root detection mechanisms to alert users of compromised execution environments. |
| Mobile Client<br>(UI / RAM)       | Information<br>Disclosure  | RAM Memory Dumping: Plaintext credentials temporarily reside in RAM while decrypted for viewing on screen. Advanced malware performs a memory dump to extract plaintext strings.                  | Exposure of sensitive plaintext credentials directly from active system memory.            | Overwrite and zero out sensitive password variables/character arrays in RAM immediately after use and leverage native secure memory boundaries where possible.                                                                                    |
| Mobile Client<br>(APK<br>Package) | Tampering                  | App Repackaging: An attacker decompiles the Project Aegis APK, removes biometric/PIN enforcement logic, repackages the app, and trick users into installing the malicious build via side-loading. | Bypass of primary authentication controls and complete compromise of local vault security. | Implement APK signature verification checks at startup and apply code obfuscation and anti-tampering guards.                                                                                                                                      |

**Note:** Threat modeling the V2 of the application is complicated due to its fully offline, minimal attack surface, encrypted, and isolated architecture. The resulting threats are advanced edge cases and limited in number.

## Vulnerability fixes

| #  | <u>v1 flaw</u>                   | <u>v2 fix</u>                                      | <u>Verified how</u>                                                                                                                                                                                                                                                                                                                        |
|----|----------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Plaintext SQLite                 | SQLCipher (AES-256) + Android Keystore-wrapped key | MobSF's dedicated "uses SQL Cipher" rule no longer fires so verified directly: manually confirmed the vault database file is unreadable ciphertext without the app's Keystore-protected key, and VaultRepository / VaultDbHelper routes every credential read/write through net.zetetic.database.sqlcipher.SQLiteDatabase. Refer. Figure 1 |
| 2  | Clipboard never clears           | 30s auto-wipe (ClipboardUtil)                      | Not MobSF-verifiable — verified manually: copy a password, wait 30s, clipboard is empty. Refer. Figure 2                                                                                                                                                                                                                                   |
| 3  | No FLAG_SECURE                   | Set via shared SecureActivity base class           | Not MobSF-verifiable (runtime window flag) — verified manually: Recents thumbnail no longer shows vault contents. Refer. Figure 3                                                                                                                                                                                                          |
| 4a | debuggable=true                  | Removed; AGP now sets it correctly per build type  | Gone from the release scan. (Still appears in the debug scan — expected, debug builds are always debuggable by design, not a flaw.) Refer. Figure 4                                                                                                                                                                                        |
| 4b | allowBackup=true                 | allowBackup="false"                                | Gone from both scans. Refer. Figure 4                                                                                                                                                                                                                                                                                                      |
| 4c | 4 activities exported=true       | All set exported="false"                           | Gone from both scans — only the androidx-library-owned ProfileInstallReceiver warning remains, which we don't control and is already permission-protected. Refer. Figure 5                                                                                                                                                                 |
| 5  | Sensitive Log.d (PIN, passwords) | Log lines deleted                                  | The generic "App logs information" finding still appears, but the raw scan data shows every flagged line points into net/zetetic/database/* (the SQLCipher library's own internal logging) — zero of our own files are listed anymore.                                                                                                     |
| 6  | Plaintext PIN, no lockout        | PBKDF2WithHmacSHA256 + salt, 5-attempt/30s lockout | Not directly MobSF-checked; verified manually (wrong PIN x5 → lockout message with countdown, confirmed working on-device). Refer. Figure 6                                                                                                                                                                                                |

```

/* JADX INFO: loaded from: classes6.dex */
public class VaultRepository {
    private static volatile VaultRepository instance;
    private final SQLiteDatabase database;

    private VaultRepository(Context context) {
        Context appContext = context.getApplicationContext();
        System.loadLibrary("sqlcipher");
        byte[] rawKey = VaultKeyProvider.getOrCreateRawKey(appContext);
        try {
            VaultDbHelper dbHelper = new VaultDbHelper(appContext, rawKey);
            this.database = dbHelper.getWritableDatabase();
        } finally {
            Arrays.fill(rawKey, (byte) 0);
        }
    }

    public static VaultRepository getInstance(Context context) {
        if (instance == null) {
            synchronized (VaultRepository.class) {
                if (instance == null) {
                    instance = new VaultRepository(context);
                }
            }
        }
        return instance;
    }

    public long insert(String accountName, String url, String username, String password) {
        ContentValues values = VaultDbHelper.toValues(accountName, url, username, password, System.currentTimeMillis());
        return this.database.insert("credentials", (String) null, values);
    }
}

```

Figure 1. Showcasing the usage of Zetetic's native SQLCipher package, the key originating from Android Keystore, and secure memory management.

```

/* JADX INFO: loaded from: classes3.dex */
public final class ClipboardUtil {
    public static final long CLEAR_DELAY_MS = 30000;

    private ClipboardUtil() {
    }

    public static void copyWithAutoWipe(Context context, Handler handler, String label, final String value) {
        final ClipboardManager clipboard = (ClipboardManager) context.getSystemService("clipboard");
        clipboard.setPrimaryClip(ClipData.newPlainText(label, value));
        handler.postDelayed(new Runnable() { // from class: com.example.projectaegis.util.ClipboardUtil$$External
            @Override // java.lang.Runnable
            public final void run() {
                ClipboardUtil.lambda$copyWithAutoWipe$0(clipboard, value);
            }
        }, 30000L);
    }

    static /* synthetic */ void lambda$copyWithAutoWipe$0(ClipboardManager clipboard, String value) {
        ClipData current = clipboard.getPrimaryClip();
        if (current != null && current.getItemCount() > 0 && value.contentEquals(current.getItemAt(0).getText()))
            clipboard.setPrimaryClip(ClipData.newPlainText("", ""));
    }
}

```

Figure 2. Showcasing the 30s auto-wipe to clear credentials from the clipboard.

```

/* JADX INFO: loaded from: classes5.dex */
public abstract class SecureActivity extends AppCompatActivity {
    @Override // androidx.fragment.app.FragmentActivity, androidx.
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        getWindow().setFlags(8192, 8192);
    }
}

```

Figure 3. Showcasing the FLAG\_SECURE bits being set in SecureActivity

```

android:allowBackup="false"

```

Figure 4. Showcasing allowBackup="false"

```

<activity
    android:label="@string/vault_title"
    android:name="com.example.projectaegis.VaultActivity"
    android:exported="false"/>
<activity
    android:name="com.example.projectaegis.EntryFormActivity"
    android:exported="false"
    android:windowSoftInputMode="adjustResize"/>
<activity
    android:name="com.example.projectaegis.CredentialDetailActivity"
    android:exported="false"/>
<activity
    android:name="com.example.projectaegis.ChangePinActivity"
    android:exported="false"/>
<provider
    android:name="androidx.startup.InitializationProvider"
    android:exported="false"

```

Figure 5. Showcasing activities setting exported="false"



```

private void registerFailedAttempt() {
    int attempts = this.prefs.getInt(KEY_FAILED_ATTEMPTS, 0) + 1;
    SharedPreferences.Editor editor = this.prefs.edit();
    if (attempts >= 5) {
        editor.putInt(KEY_FAILED_ATTEMPTS, 0);
        editor.putLong(KEY_LOCKOUT_UNTIL, System.currentTimeMillis() + 30000);
    } else {
        editor.putInt(KEY_FAILED_ATTEMPTS, attempts);
    }
    editor.apply();
}

```

Figure 6. Showcasing the application locking out after five attempts

## Reflection

Static analysis tools like MobSF are invaluable for catching manifest configurations, obsolete SDK targets, and accidental hardcoded strings. However, relying solely on automated security scores presents a distorted view of actual security.

MobSF v1 security score: **42**

MobSF v2 debug security score: **38**

MobSF v2 release (unsigned) security score: **45**

Every vulnerability intentionally planted in v1 was successfully addressed in v2. Where the scanner was blind to the fix, security was confirmed through direct and dynamic verification utilizing MobSF, code review, and realtime ADB. Even though the later implementation of the application is leagues stronger security wise, it received a relatively unchanged score than the previous version for a number of factors.

The scan reported several security findings from third-party libraries, false positives, scanner rule gaps, and structural blind spots regarding runtime behavior:

- **Bundled Third-Party Code:** Warnings for raw SQL queries and sensitive logging still flagged in v2, but 100% of these lines came from inside the SQLCipher library (net/zetetic/database/\*), not our application code.

|   |                                                                                                                                                                                                                  |         |                                                                                                                                            |                                                                                                       |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 2 | <a href="#">App uses SQLite Database and execute raw SQL query. Untrusted user input in raw SQL queries can cause SQL Injection. Also sensitive information should be encrypted and written to the database.</a> | warning | CWE: CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')<br>OWASP Top 10: M7: Client Code Quality | net/zetetic/database/DatabaseUtil<br>s.java<br>net/zetetic/database/sqlcipher/SQ<br>LiteDatabase.java |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|

- Outdated Scanner Rules: MobSF looks strictly for the old `net.sqlcipher.*` package path. When we updated to `net.zetetic.database.sqlcipher.*` for modern 16KB page-size support, the scanner failed to recognize our active AES-256 database encryption entirely.
- Keyword Matching False Positives: Flags for "hardcoded secrets" turned out to be basic string literals—like database table names ("aegis\_vault.db") and a word in a developer comment ("key" in `VaultKeyProvider.java`).
- Blindness to Runtime Defenses: Passive static tools can't watch the app run, so MobSF completely missed our dynamic protections:

- Screen Recording/Recents Blocking: Enforced at runtime via `FLAG_SECURE` in `SecureActivity`.

```
/* JADX INFO: loaded from: classes5.dex */
public abstract class SecureActivity extends AppCompatActivity {
    @Override // androidx.fragment.app.FragmentActivity, androidx.
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        getWindow().setFlags(8192, 8192);
    }
}
```

- Auto-Wiping Clipboard: Cleared in memory after 30 seconds via background tasks in `ClipboardUtil`.

```
public static void copyWithAutoWipe(Context context, Handler handler, String label, final String value) {
    final ClipboardManager clipboard = (ClipboardManager) context.getSystemService("clipboard");
    clipboard.setPrimaryClip(ClipData.newPlainText(label, value));
    handler.postDelayed(new Runnable() { // from class: com.example.projectaegis.util.ClipboardUtil$$Extern
        @Override // java.lang.Runnable
        public final void run() {
            ClipboardUtil.lambda$copyWithAutoWipe$0(clipboard, value);
        }
    }, 30000L);
}
```

Ultimately, MobSF is great for preliminary checks for surface-level configuration errors, but cannot fully replace manual code inspection and dynamic testing.

There are also known limitations and risks that should be highlighted:

- Firebase and cloud synchronization features were deliberately excluded from the project scope. Keeping the application fully offline ensures that all local storage vulnerabilities remain directly testable and observable within static analysis bounds, minimizing external attack vectors.
- MobSF flags the minimum SDK setting (`minSdk=24`) because it allows installation on Android 7.0 devices, which no longer receive security patches.

- Migrating from Version 1 (plaintext SQLite) to Version 2 (SQLCipher encrypted storage) changes the underlying file structure, rendering the legacy database unreadable by Version 2 binaries without a custom migration process. For the scope of this capstone project, requiring a clean app re-installation between versions was accepted.